GPU-Accelerated Heterogeneous String Pattern Matching

Abigale Kim, Xiangyao Yu
University of Wisconsin—Madison
abigale@cs.wisc.edu

Computer Sciences
SCHOOL OF COMPUTER, DATA & INFORMATION SCIENCES
UNIVERSITY OF WISCONSIN–MADISON

Motivation

- GPUs provide significant performance advantages over CPUs for large scale string processing
- Naïve parallelization approach maps each string to a thread or warp (32 threads), but this approach yields thread divergence and hardware under-utilization because short strings take less time to process, but must wait for longer strings to finish
- Our approach targets workloads with highly skewed string length distributions

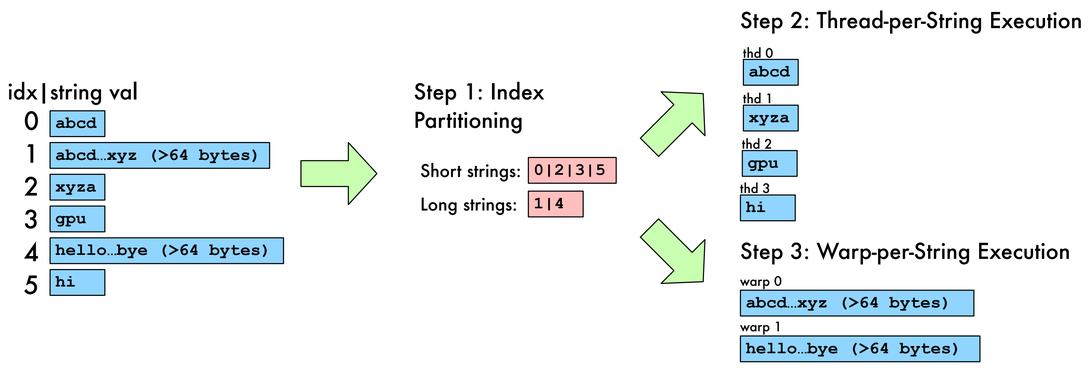
Heterogeneous Pattern Matching Design

Observations

- Thread-per-string implementation is better for shorter strings due to more efficient resource allocation, warp-per-string implementation is better for longer strings due to cache-aligned memory accesses

Design

1. Index partitioning: String indices are partitioned based on a length threshold of 64 bytes
2. Thread-per-String Execution: Short strings (<64 bytes) are processed by a thread-per-string kernel, where each CUDA thread scans its assigned string for the target pattern
3. Warp-per-String Execution: Long strings (≥64 bytes) are processed by a warp-per-string kernel, where each CUDA warp scans a cache line (128 bytes) per iteration, improving cache hit rates and memory throughput



Implementation

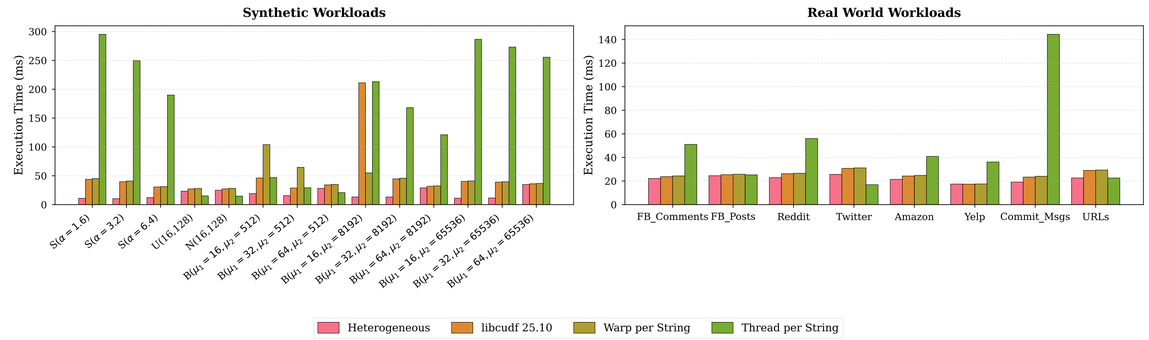
- Built on NVIDIA's cuDF, a high-performance GPU dataframe library widely used in the RAPID's ecosystem
- Used Thrust's unstable partition (outperforms stable partitioning in benchmarks)
- Submitted a PR (#21917) to the `rapidsai/cudf` repository

Evaluation

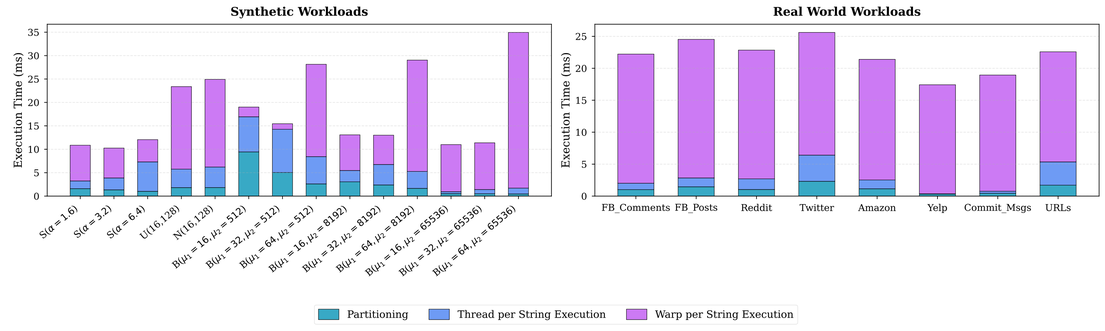
Specifications

- NVIDIA RTX A5000 GPU, 24 GB GPU memory, 768 GB/s memory bandwidth
- Synthetic (skewed, normal, uniform, bimodal distr.) & real-world datasets (Facebook, Reddit, Twitter posts/comments, Amazon/Yelp reviews, GitHub commit messages, common URLs)

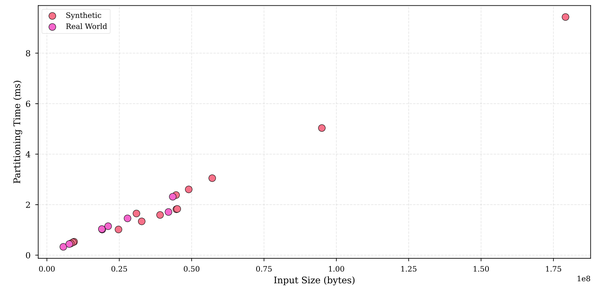
Overall Performance



Heterogeneous Implementation Breakdown



Input Size vs. Partitioning Time



Discussion

- Max 3.9x speedup on synthetic workloads
- Max 1.3x speedup on real-world workloads
- Heterogeneous execution is very effective when the length distribution has high skew
- Partitioning overhead scales linearly to input size, the cost is much smaller compared to the speedup gained during execution